

Design and Analysis of Software Systems

Assignment 2 Part 3 : Black Box API Testing

Meet Parekh (2024101122)

1 Introduction

To perform black-box testing on the QuickCart REST API, especially when I didn't have access to the internal implementation, I designed test cases based solely on the API documentation provided. I used Python with `pytest` and the `requests` library to automate the testing process.

The test suite contains 159 test cases covering various aspects of the API including valid requests, invalid inputs, missing fields, wrong data types, and boundary values. Out of these, 130 tests passed and 29 tests failed, revealing several bugs in the API implementation.

2 Test Case Design

I organized the test cases into logical categories based on the API endpoints and the type of validation being performed.

2.1 Global Header Tests

These tests verify that the API correctly handles the required headers.

Test Case	Input	Expected Output	Justification
Missing X-Roll-Number	Request without X-Roll-Number header	401 Unauthorized	API should reject requests without authentication
Invalid roll number with letters	X-Roll-Number: "abc123"	400 Bad Request	Roll number should be numeric only
Invalid roll number with symbols	X-Roll-Number: "123!@#"	400 Bad Request	Roll number should not contain special characters
Missing X-User-ID for user endpoint	GET /cart without X-User-ID	400 Bad Request	User endpoints require user identification
Invalid X-User-ID	X-User-ID: "invalid"	400 Bad Request	User ID should be a valid number
Negative X-User-ID	X-User-ID: "-1"	400 Bad Request	User ID cannot be negative
Admin endpoint without X-User-ID	GET /admin/products	200 OK	Admin endpoints should not require X-User-ID

2.2 Profile Tests

These tests verify user profile operations.

Test Case	Input	Expected Output	Justification
Get profile success	GET /profile	200 OK with profile data	Basic functionality test
Update with valid name	name: "John Doe", phone: "1234567890"	200 OK	Valid data should be accepted
Name too short	name: "J" (1 char)	400 Bad Request	Name must be at least 2 characters
Name too long	name: 51 characters	400 Bad Request	Name must not exceed 50 characters
Name boundary min	name: "Jo" (2 chars)	200 OK	Boundary test for minimum length
Name boundary max	name: 50 characters	200 OK	Boundary test for maximum length
Phone not 10 digits	phone: "123456789" (9 digits)	400 Bad Request	Phone must be exactly 10 digits
Phone too long	phone: "12345678901" (11 digits)	400 Bad Request	Phone must not exceed 10 digits
Phone with letters	phone: "123456789a"	400 Bad Request	Phone should contain only digits

2.3 Address Tests

These tests verify address management operations.

Test Case	Input	Expected Output	Justification
Get addresses success	GET /addresses	200 OK	Basic functionality test
Add valid address	label: HOME, valid street, city, pincode	200/201	Valid data should be accepted
Invalid label	label: "INVALID"	400 Bad Request	Only HOME, OFFICE, OTHER are valid
Label OFFICE	label: "OFFICE"	200/201	OFFICE is a valid label
Label OTHER	label: "OTHER"	200/201	OTHER is a valid label
Street too short	street: "123" (3 chars)	400 Bad Request	Street must be at least 5 characters
Street too long	street: 101 characters	400 Bad Request	Street must not exceed 100 characters
City too short	city: "T" (1 char)	400 Bad Request	City must be at least 2 characters
City too long	city: 51 characters	400 Bad Request	City must not exceed 50 characters

Test Case	Input	Expected Output	Justification
Invalid pincode length	pincode: "12345" (5 digits)	400 Bad Request	Pincode must be exactly 6 digits
Pincode with letters	pincode: "12345a"	400 Bad Request	Pincode should contain only digits
Delete nonexistent address	DELETE /addresses/999999	404 Not Found	Should return 404 for invalid ID

2.4 Product Tests

These tests verify product listing and retrieval.

Test Case	Input	Expected Output	Justification
Get products success	GET /products	200 OK	Basic functionality test
Get single product	GET /products/{id}	200 OK with product	Should return product details
Nonexistent product	GET /products/999999	404 Not Found	Should return 404 for invalid ID
Filter by category	?category=electronics	200 OK	Category filtering should work
Search by name	?search=test	200 OK	Search functionality should work
Sort by price ascending	?sort=price_asc	200 OK	Sorting should work
Sort by price descending	?sort=price_desc	200 OK	Sorting should work

2.5 Cart Tests

These tests verify shopping cart operations.

Test Case	Input	Expected Output	Justification
Get cart success	GET /cart	200 OK	Basic functionality test
Add valid item	product_id: valid, quantity: 1	200 OK	Valid data should be accepted
Quantity zero	quantity: 0	400 Bad Request	Zero quantity is invalid
Negative quantity	quantity: -1	400 Bad Request	Negative quantity is invalid
Nonexistent product	product_id: 999999	404 Not Found	Should return 404 for invalid product
Same product twice	Add same product twice	Accumulates quantity	Quantity should accumulate
Exceeding stock	quantity > stock	400 Bad Request	Cannot add more than available

Test Case	Input	Expected Output	Justification
Subtotal calculation	quantity: 2	subtotal = 2 * price	Math should be correct
Total calculation	Multiple items	sum of subtotals	Total should be sum of all
Update cart item	POST /cart/update	200 OK	Update should work
Update to zero	quantity: 0	400 Bad Request	Cannot update to zero
Remove item	POST /cart/remove	200 OK	Remove should work
Remove nonexistent	product_id: 999999	404 Not Found	Should return 404
Clear cart	DELETE /cart/clear	200 OK	Clear should work

2.6 Checkout Tests

These tests verify the checkout process.

Test Case	Input	Expected Output	Justification
Empty cart checkout	POST /checkout with empty cart	400 Bad Request	Cannot checkout empty cart
Invalid payment method	payment_method: "BITCOIN"	400 Bad Request	Only COD, CARD, WALLET valid
COD valid	payment_method: "COD", total <5000	200 OK	COD allowed under 5000
COD over 5000	payment_method: "COD", total >5000	400 Bad Request	COD not allowed over 5000
Wallet payment	payment_method: "WALLET"	200 OK	Wallet payment should work
Card payment	payment_method: "CARD"	200 OK	Card payment should work
GST applied once	Check total with GST	total = subtotal * 1.05	GST should be exactly 5%

2.7 Wallet Tests

These tests verify wallet operations.

Test Case	Input	Expected Output	Justification
Get wallet balance	GET /wallet	200 OK with balance	Basic functionality test
Add zero amount	amount: 0	400 Bad Request	Zero amount is invalid
Add negative amount	amount: -100	400 Bad Request	Negative amount is invalid

Test Case	Input	Expected Output	Justification
Add over 100000	amount: 100001	400 Bad Request	Amount must not exceed 100000
Add boundary max	amount: 100000	200 OK	Boundary test for max
Add valid amount	amount: 500	200 OK	Valid amount should work
Pay exceeds balance	amount > balance	400 Bad Request	Cannot pay more than balance
Pay zero	amount: 0	400 Bad Request	Zero payment is invalid
Pay negative	amount: -100	400 Bad Request	Negative payment is invalid
Exact deduction	Pay 100 from balance	balance - 100	Deduction should be exact

2.8 Orders Tests

These tests verify order management.

Test Case	Input	Expected Output	Justification
Get orders success	GET /orders	200 OK	Basic functionality test
Nonexistent order	GET /orders/999999	404 Not Found	Should return 404 for invalid ID
Cancel nonexistent	POST /orders/999999/cancel	404 Not Found	Should return 404 for invalid ID
Cancel delivered order	Cancel order with DELIVERED status	400 Bad Request	Delivered orders cannot be cancelled
Invoice totals	GET /orders/{id}/invoice	GST = subtotal * 0.05	GST should be 5% of subtotal
Cancel restores stock	Cancel order	Stock increases	Stock should be restored on cancel

2.9 Reviews Tests

These tests verify product review operations.

Test Case	Input	Expected Output	Justification
Get product reviews	GET /products/{id}/reviews	200 OK	Basic functionality test
Add valid review	rating: 4, comment: "Good"	200/201	Valid review should be accepted
Rating zero	rating: 0	400 Bad Request	Rating must be 1-5

Test Case	Input	Expected Output	Justification
Rating six	rating: 6	400 Bad Request	Rating must be 1-5
Negative rating	rating: -1	400 Bad Request	Negative rating is invalid
Rating boundary min	rating: 1	200/201	Minimum valid rating
Rating boundary max	rating: 5	200/201	Maximum valid rating
Empty comment	comment: ""	400 Bad Request	Comment cannot be empty
Comment too long	comment: 201 chars	400 Bad Request	Comment max is 200 chars
Comment boundary max	comment: 200 chars	200/201	Boundary test for max

2.10 Support Ticket Tests

These tests verify support ticket operations.

Test Case	Input	Expected Output	Justification
Get tickets success	GET /support/tickets	200 OK	Basic functionality test
Create valid ticket	subject: "Test", message: "Help"	200/201	Valid ticket should be created
Subject too short	subject: "Hi" (2 chars)	400 Bad Request	Subject must be at least 5 chars
Subject too long	subject: 101 chars	400 Bad Request	Subject max is 100 chars
Subject boundary min	subject: 5 chars	200/201	Boundary test for min
Subject boundary max	subject: 100 chars	200/201	Boundary test for max
Empty message	message: ""	400 Bad Request	Message cannot be empty
Message too long	message: 501 chars	400 Bad Request	Message max is 500 chars
Message boundary max	message: 500 chars	200/201	Boundary test for max
Starts with OPEN status	Create new ticket	status: "OPEN"	New tickets start as OPEN
OPEN to IN_PROGRESS	Update status	200 OK	Valid status transition
CLOSED to OPEN	Update status	400 Bad Request	Cannot reopen closed ticket

2.11 Missing Field Tests

These tests verify that required fields are properly validated.

Test Case	Input	Expected Output	Justification
Cart add missing product_id	Only quantity provided	400 Bad Request	product_id is required
Cart add missing quantity	Only product_id provided	400 Bad Request	quantity is required
Checkout missing payment	Empty payload	400 Bad Request	payment_method is required
Address missing label	No label field	400 Bad Request	label is required
Address missing street	No street field	400 Bad Request	street is required
Address missing city	No city field	400 Bad Request	city is required
Address missing pincode	No pincode field	400 Bad Request	pincode is required
Review missing rating	Only comment	400 Bad Request	rating is required
Review missing comment	Only rating	400 Bad Request	comment is required
Ticket missing subject	Only message	400 Bad Request	subject is required
Ticket missing message	Only subject	400 Bad Request	message is required
Coupon missing code	Empty payload	400 Bad Request	code is required

2.12 Data Type Tests

These tests verify that wrong data types are rejected.

Test Case	Input	Expected Output	Justification
String quantity	quantity: "five"	400 Bad Request	Quantity must be integer
Float quantity	quantity: 1.5	400 Bad Request	Quantity must be integer
String wallet amount	amount: "hundred"	400 Bad Request	Amount must be number
String rating	rating: "five"	400 Bad Request	Rating must be integer

2.13 Edge Case Tests

These tests verify handling of edge cases.

Test Case	Input	Expected Output	Justification
Empty JSON body	{ } to /cart/add	400 Bad Request	Empty payload should be rejected
Null values	product_id: null, quantity: null	400/404	Null values should be handled
Very large quantity	quantity: 999999999	400 Bad Request	Unreasonable values rejected
XSS in search	search=<script>alert(1)</script>	200 OK	Should handle safely
SQL injection	search='; DROP TABLE products; --	200 OK	Should handle safely

3 Test Results Summary

- **Total Tests:** 159
- **Passed:** 130
- **Failed:** 29
- **Pass Rate:** 81.76%

The failing tests revealed several bugs in the API which are documented in the Bug Report section below.

4 Bug Report

During testing, I discovered the following bugs in the QuickCart API. Each bug includes the endpoint tested, request details, expected behavior based on documentation, and actual behavior observed.

4.1 Bug 1: Profile Accepts Phone with Letters

Endpoint: PUT /profile

Request:

Method: PUT

URL: http://localhost:8080/api/v1/profile

Headers:

- X-Roll-Number: 2024101122
- X-User-ID: 1

Body: {"name": "John Doe", "phone": "123456789a"}

Expected Result: 400 Bad Request - The phone field should only accept numeric characters. According to the documentation, phone must be exactly 10 digits.

Actual Result: 200 OK - The API accepts the phone number containing a letter and updates the profile successfully.

Failed Tests: test_update_profile_phone_with_letters, test_bug_profile_accepts_phone_with_letters

4.2 Bug 2: Address Accepts Invalid Pincode Length

Endpoint: POST /addresses

Request:

Method: POST

URL: http://localhost:8080/api/v1/addresses

Headers:

- X-Roll-Number: 2024101122
- X-User-ID: 1

Body: {"label": "HOME", "street": "123 Test Street",
"city": "TestCity", "pincode": "12345", "is_default": false}

Expected Result: 400 Bad Request - The pincode must be exactly 6 digits according to the documentation.

Actual Result: 200 OK - The API accepts 5-digit and 7-digit pincodes without validation.

Failed Tests: test_add_address_invalid_pincode_length, test_bug_address_accepts_5_digit_pincode, test_bug_address_accepts_7_digit_pincode

4.3 Bug 3: Address Accepts Missing Pincode

Endpoint: POST /addresses

Request:

Method: POST

URL: http://localhost:8080/api/v1/addresses

Headers:

- X-Roll-Number: 2024101122
- X-User-ID: 1

Body: {"label": "HOME", "street": "123 Test Street", "city": "Test City"}

Expected Result: 400 Bad Request - The pincode field is required according to the documentation.

Actual Result: 200 OK - The API creates an address without a pincode.

Failed Tests: test_address_missing_pincode, test_bug_address_no_pincode_accepted

4.4 Bug 4: Cart Accepts Zero Quantity

Endpoint: POST /cart/add

Request:

Method: POST

URL: http://localhost:8080/api/v1/cart/add

Headers:

- X-Roll-Number: 2024101122
- X-User-ID: 1

Body: {"product_id": 1, "quantity": 0}

Expected Result: 400 Bad Request - Quantity should be a positive integer greater than zero.

Actual Result: 200 OK - The API accepts zero quantity and adds it to the cart.

Failed Tests: test_add_item_quantity_zero_returns_400, test_bug_cart_accepts_zero_quantity

4.5 Bug 5: Cart Subtotal Calculation Incorrect

Endpoint: GET /cart (after adding items)

Request:

Method: POST /cart/add then GET /cart

URL: http://localhost:8080/api/v1/cart

Headers:

- X-Roll-Number: 2024101122
- X-User-ID: 1

Setup: Add product_id 1 with quantity 2 (product price = 120)

Expected Result: Item subtotal should be 240 ($2 * 120 = 240$)

Actual Result: Item subtotal returned as -16

Failed Tests: test_bug_cart_subtotal_incorrect

4.6 Bug 6: Cart Total Calculation Incorrect

Endpoint: GET /cart (after adding items)

Request:

Method: POST /cart/add then GET /cart

URL: http://localhost:8080/api/v1/cart

Headers:

- X-Roll-Number: 2024101122
- X-User-ID: 1

Setup: Add product 1 (price 120) and product 2 (price 130)

Expected Result: Cart total should be 250 ($120 + 130$)

Actual Result: Cart total returned as 120

Failed Tests: test_bug_cart_total_incorrect

4.7 Bug 7: Cart Add Missing Quantity Accepted

Endpoint: POST /cart/add

Request:

Method: POST

URL: http://localhost:8080/api/v1/cart/add

Headers:

- X-Roll-Number: 2024101122
- X-User-ID: 1

Body: {"product_id": 1}

Expected Result: 400 Bad Request - The quantity field is required.

Actual Result: 200 OK - The API adds the item without specifying quantity.

Failed Tests: test_cart_add_missing_quantity, test_bug_cart_add_no_quantity_accepted

4.8 Bug 8: Cart Add Missing Product ID Returns 404

Endpoint: POST /cart/add

Request:

Method: POST

URL: http://localhost:8080/api/v1/cart/add

Headers:

- X-Roll-Number: 2024101122
- X-User-ID: 1

Body: {"quantity": 1}

Expected Result: 400 Bad Request - Missing required field should return 400.

Actual Result: 404 Not Found - The API returns 404 instead of 400 for missing product_id.

Failed Tests: test_cart_add_missing_product_id, test_bug_cart_add_no_product_id_returns_404

4.9 Bug 9: Empty JSON Body Returns 404

Endpoint: POST /cart/add

Request:

Method: POST

URL: http://localhost:8080/api/v1/cart/add

Headers:

- X-Roll-Number: 2024101122
- X-User-ID: 1

Body: {}

Expected Result: 400 Bad Request - Empty payload should be rejected with 400.

Actual Result: 404 Not Found - The API returns 404 instead of 400.

Failed Tests: test_empty_json_body, test_bug_empty_json_returns_404

4.10 Bug 10: Wallet Precision Error

Endpoint: POST /wallet/add and POST /wallet/pay

Request:

Method: POST /wallet/add, then POST /wallet/pay

URL: http://localhost:8080/api/v1/wallet/add

Headers:

- X-Roll-Number: 2024101122
- X-User-ID: 1

Operations: Add 100, then pay 50

Expected Result: Balance should change exactly by the amounts specified (initial + 100 - 50)

Actual Result: Balance has floating-point precision errors. Expected 111534.79, got 111533.98999999999 (off by 0.8)

Failed Tests: test_wallet_exact_deduction, test_bug_wallet_precision_error

4.11 Bug 11: Can Cancel Delivered Orders

Endpoint: POST /orders/{id}/cancel

Request:

Method: POST

URL: http://localhost:8080/api/v1/orders/{order_id}/cancel

Headers:

- X-Roll-Number: 2024101122
- X-User-ID: 1

Setup: Find an order with status "DELIVERED"

Expected Result: 400 Bad Request - Delivered orders should not be cancellable.

Actual Result: 200 OK - The API allows cancelling orders that have already been delivered.

Failed Tests: test_cancel_delivered_order_returns_400

4.12 Bug 12: Invoice GST Calculation Incorrect

Endpoint: GET /orders/{id}/invoice

Request:

Method: GET

URL: http://localhost:8080/api/v1/orders/{order_id}/invoice

Headers:

- X-Roll-Number: 2024101122
- X-User-ID: 1

Expected Result: GST should be 5% of subtotal. For subtotal of 120, GST should be 6.0

Actual Result: GST returned as 12.3 instead of 6.0 (approximately double)

Failed Tests: test_order_invoice_shows_correct_totals, test_bug_invoice_gst_calculation

4.13 Bug 13: Reviews Accepted for Nonexistent Products

Endpoint: POST /products/{id}/reviews

Request:

Method: POST

URL: http://localhost:8080/api/v1/products/999999/reviews

Headers:

- X-Roll-Number: 2024101122
- X-User-ID: 1

Body: {"rating": 4, "comment": "Test"}

Expected Result: 404 Not Found - Cannot review a product that doesn't exist.

Actual Result: 200 OK - The API accepts reviews for nonexistent products.

Failed Tests: test_bug_review_nonexistent_product

4.14 Bug 14: Average Rating Rounded to Integer

Endpoint: GET /products/{id}/reviews

Request:

Method: GET

URL: http://localhost:8080/api/v1/products/1/reviews

Headers:

- X-Roll-Number: 2024101122
- X-User-ID: 1

Expected Result: average_rating should be a float with decimal precision (e.g., 3.5)

Actual Result: average_rating is returned as a rounded integer (e.g., 3 instead of 3.5)

Failed Tests: test_bug_average_rating_rounded

4.15 Bug 15: Ticket Status Can Skip from OPEN to CLOSED

Endpoint: PUT /support/tickets/{id}

Request:

Method: PUT

URL: http://localhost:8080/api/v1/support/tickets/{ticket_id}

Headers:

- X-Roll-Number: 2024101122
- X-User-ID: 1

Body: {"status": "CLOSED"}

Setup: Ticket with status "OPEN"

Expected Result: 400 Bad Request - Status should follow valid transitions (OPEN -> IN_PROGRESS -> CLOSED)

Actual Result: 200 OK - The API allows changing status directly from OPEN to CLOSED, skipping IN_PROGRESS.

Failed Tests: test_bug_ticket_open_to_closed_directly

4.16 Bug 16: Ticket Accepts Invalid Status Values

Endpoint: PUT /support/tickets/{id}

Request:

Method: PUT

URL: http://localhost:8080/api/v1/support/tickets/{ticket_id}

Headers:

- X-Roll-Number: 2024101122
- X-User-ID: 1

Body: {"status": "INVALID_STATUS"}

Expected Result: 400 Bad Request - Only valid status values (OPEN, IN_PROGRESS, CLOSED) should be accepted.

Actual Result: 200 OK - The API accepts arbitrary invalid status values.

Failed Tests: test_bug_ticket_accepts_invalid_status

4.17 Bug 17: Address API Requires Undocumented Field

Endpoint: POST /addresses

Request:

Method: POST

URL: http://localhost:8080/api/v1/addresses

Headers:

- X-Roll-Number: 2024101122
- X-User-ID: 1

Body: {"label": "HOME", "street": "123 Test Street",
"city": "Test City", "pincode": "123456"}

Expected Result: 200/201 Created - All documented required fields are provided.

Actual Result: 400 Bad Request - The API requires the `is_default` field which is not documented as a required input field.

Failed Tests: test_add_address_valid, test_add_address_label_office, test_add_address_label_other

5 Conclusion

Through this black-box testing exercise, I identified 17 distinct bugs in the QuickCart API. The most critical issues are related to business logic errors in cart and invoice calculations, which could directly impact financial transactions. Input validation bugs are also common, where the API accepts invalid data that should be rejected. These findings highlight the importance of thorough API testing before deployment.